Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo de un Red-Black Tree y el centinela $T.nil$
- Rotaciones con punteros al padre
- RB-INSERTAR: insertar rojo y arreglar mirando al tío
- RB-ELIMINAR: el negro extra y los cuatro casos del hermano

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase 16

Un Red-Black Tree tiene altura a lo más $2\lg(n+1)$. El nodo nuevo entra rojo y, si su padre también es rojo, se mira al tío.

En la clase 17 implementamos el AVL de forma recursiva, sin puntero al padre. ¿Por qué el Red-Black se implementa distinto?

Porque sus arreglos miran hacia arriba

Tío, abuelo, hermano: los casos dependen de parientes que no están debajo del nodo. Con *padre* se arregla subiendo en un ciclo, como en el ELIMINAR del BST de la clase 13.

El nodo y el centinela

El nodo, y un centinela

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Campos

v.clave, *v.color* (ROJO o NEGRO), *v.izq*, *v.der* y *v.padre*.

Las reglas hablan del color de las hojas NIL, y los arreglos preguntan por el color del tío o del hermano, que pueden no existir. ¿Hay que preguntar "¿es nulo?" en cada línea?

No: un único nodo centinela, *T.nil*

Un nodo NEGRO que hace de todas las hojas NIL y del padre de la raíz. Donde el BST tenía NULO, ahora va *T.nil*, y preguntar *T.nil.color* siempre funciona: es NEGRO.

Rotar-Izquierda, ahora con padre

Algoritmo 1: ROTAR-IZQUIERDA(T, x)

```
y ← x.der
x.der ← y.izq
si y.izq ≠ T.nil entonces y.izq.padre ← x
y.padre ← x.padre
si x.padre = T.nil entonces
    T.raiz ← y
si no, si x = x.padre.izq entonces
    x.padre.izq ← y
en otro caso
    x.padre.der ← y
y.izq ← x,  x.padre ← y
```

Insertar

RB-Insertar

Algoritmo 2: RB-Insertar(T, z)

$y \leftarrow T.nil$, $x \leftarrow T.raiz$
mientras $x \neq T.nil$ **hacer**
    $y \leftarrow x$
    si $z.clave < x.clave$ **entonces** $x \leftarrow x.izq$ **en otro caso** $x \leftarrow x.der$
$z.padre \leftarrow y$
si $y = T.nil$ **entonces**
    $T.raiz \leftarrow z$
si no, si $z.clave < y.clave$ **entonces**
    $y.izq \leftarrow z$
en otro caso $y.der \leftarrow z$
$z.izq \leftarrow T.nil$, $z.der \leftarrow T.nil$, $z.color \leftarrow$ ROJO
RB-Insertar-Arreglar(T, z)

Dos rojos seguidos: tres casos

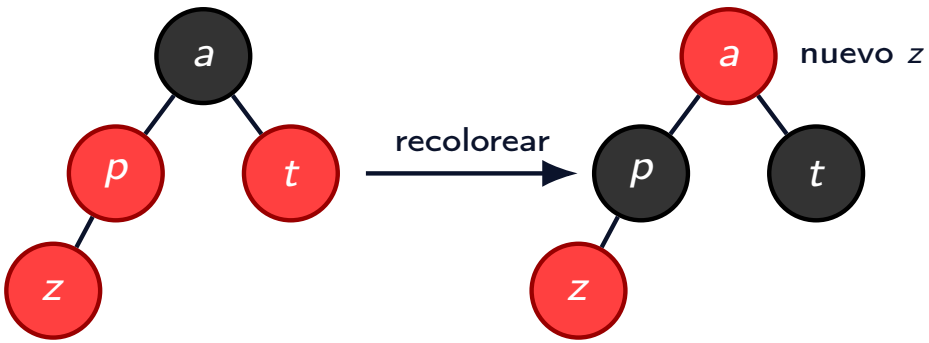
UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Situación

z es rojo y su padre p también. Como la raíz es negra, p no es la raíz: existe el abuelo a, y es negro. Sea t el tío (el otro hijo de a). Supongamos que p es hijo **izquierdo** de a; el otro lado es el espejo.

Caso	**Cuándo**	**Qué se hace**
1	t es rojo	recolorear y subir a a
2	t negro, z hijo derecho	rotar en p: se vuelve caso 3
3	t negro, z hijo izquierdo	recolorear y rotar en a: fin

Caso 1: el tío es rojo

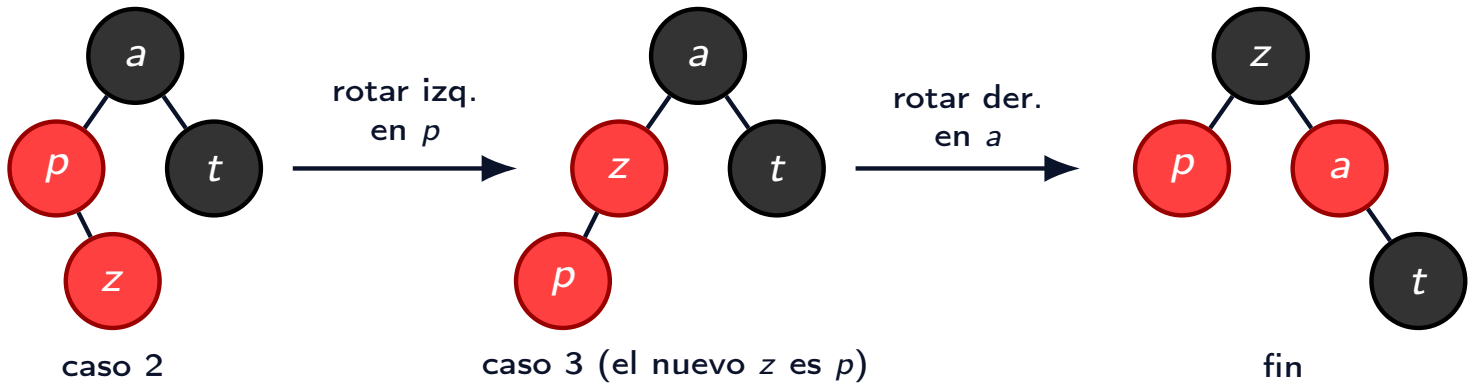


$z.padre.color \leftarrow \text{NEGRO}, \quad t.color \leftarrow \text{NEGRO}$

$z.padre.padre.color \leftarrow \text{ROJO}, \quad z \leftarrow z.padre.padre$

El abuelo le pasa su negro a sus dos hijos: todo camino que pasa por *a* sigue con los mismos negros. Pero ahora *a* es rojo: si su padre también, el problema subió dos niveles, y el ciclo sigue desde ahí.

Casos 2 y 3: el tío es negro



---

si $z = z.padre.der$ **entonces**

$z \leftarrow z.padre$, ROTAR-IZQUIERDA(T, z) // caso 2

$z.padre.color \leftarrow$ NEGRO, $z.padre.padre.color \leftarrow$ ROJO // caso 3

ROTAR-DERECHA$(T, z.padre.padre)$

Tras el caso 3, el que quedó arriba es negro: no hay dos rojos seguidos, los negros por camino no cambiaron, y el ciclo termina.

RB-Insertar-Arreglar

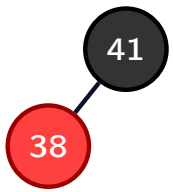
Algoritmo 3: RB-Insertar-Arreglar(T, z)

```
mientras z.padre.color = ROJO hacer
    si z.padre = z.padre.padre.izq entonces
        t ← z.padre.padre.der                                  // el tío
        si t.color = ROJO entonces
            caso 1
        en otro caso
            si z = z.padre.der entonces caso 2
            caso 3
    en otro caso
        lo mismo, intercambiando izq y der
T.raiz.color ← NEGRO
```

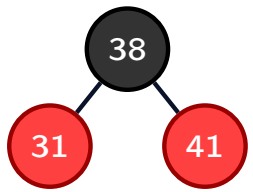
Un ejemplo: insertar $41, 38, 31, 12, 19, 8$



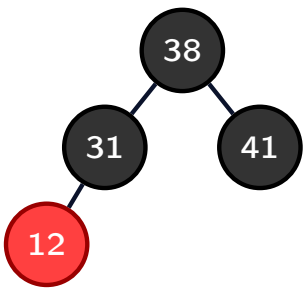
41



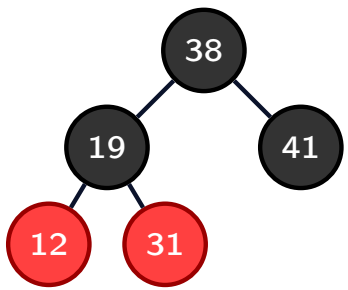
+38



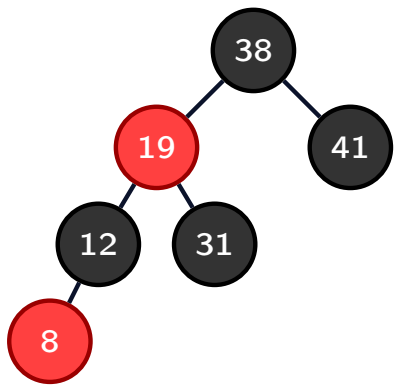
+31: caso 3



+12: caso 1



+19: casos 2 y 3



+8: caso 1

Con 19: su padre 12 es rojo, el tío (hijo derecho de 31) es $T.nil$, negro, y 19 es hijo derecho: caso 2, luego caso 3.

¿Cuánto cuesta insertar?

El caso 1 puede repetirse. ¿Cuántas veces, y cuántas rotaciones hay en total?

$O(\lg n)$ vueltas, y a lo más 2 rotaciones

Cada caso 1 sube z dos niveles, así que se repite a lo más $O(\lg n)$ veces, y solo recolorea. Los casos 2 y 3 aparecen a lo más una vez, justo antes de terminar: una o dos rotaciones.

RB-Insertar cuesta $O(\lg n)$: bajar por la altura, y subir recoloreando.

Eliminar

RB-Transplantar

Algoritmo 5: RB-TRANSPLANTAR(T, u, v)

```
si u.padre = T.nil entonces
    T.raiz ← v
si no, si u = u.padre.izq entonces
    u.padre.izq ← v
en otro caso u.padre.der ← v
v.padre ← u.padre
```

Casi el TRANSPLANTAR de la clase 13, con una diferencia: la última línea se hace **siempre**, aunque v sea $T.nil$. RB-ELIMINAR va a necesitar saber quién es el padre de ese centinela.

RB-Eliminar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 6: RB-Eliminar(T, z)

```
y ← z,  c ← y.color                                   // c: color original de y
si z.izq = T.nil entonces
    x ← z.der,  RB-Transplantar(T, z, z.der)
si no, si z.der = T.nil entonces
    x ← z.izq,  RB-Transplantar(T, z, z.izq)
en otro caso
    y ← Mínimo(z.der),  c ← y.color,  x ← y.der
    si y.padre = z entonces
        x.padre ← y
    en otro caso
        RB-Transplantar(T, y, y.der),  y.der ← z.der,  y.der.padre ← y
    RB-Transplantar(T, z, y),  y.izq ← z.izq,  y.izq.padre ← y
    y.color ← z.color
si c = negro entonces RB-Eliminar-Arreglar(T, x)
```

¿Qué se rompe al eliminar?

Si y era rojo, ¿se rompe algo?

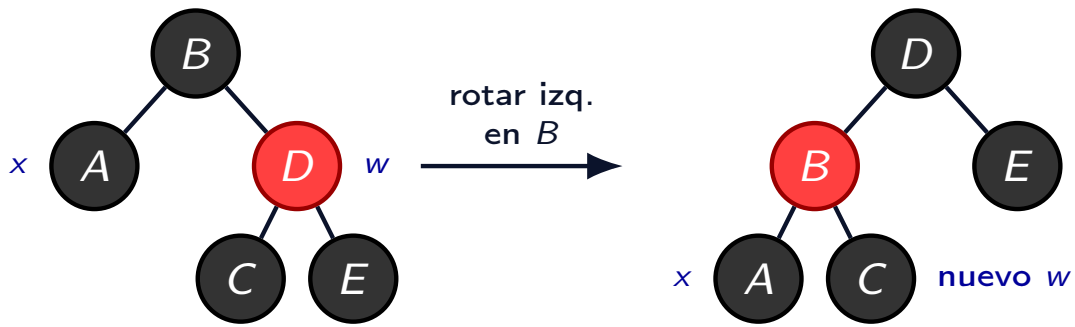
No

Ningún camino pierde negros, y no aparecen dos rojos seguidos. Por eso el arreglo solo se llama si $c = \text{NEGRO}$.

Si y era negro: el negro extra

Los caminos que pasaban por y ahora tienen un negro menos. Pensemos que x carga un negro **extra**. Si x es rojo, basta pintarlo de negro. Si x es negro (y no es la raíz), queda "doblemente negro": hay que empujar ese negro hacia arriba, o absorberlo con rotaciones, mirando al **hermano** w de x. Hay cuatro casos.

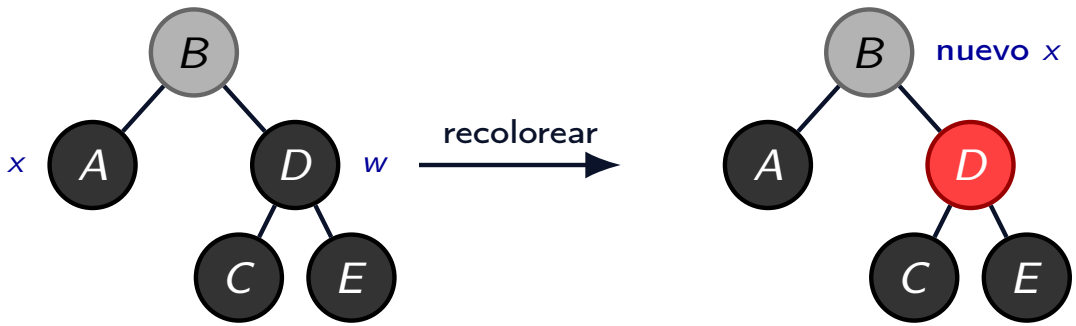
Caso 1: el hermano w es rojo



$w.color \leftarrow$ NEGRO, $x.padre.color \leftarrow$ ROJO
ROTAR-IZQUIERDA$(T, x.padre)$, $w \leftarrow x.padre.der$

No resuelve nada todavía, pero deja a x con un hermano **negro**: pasamos a los casos 2, 3 o 4.

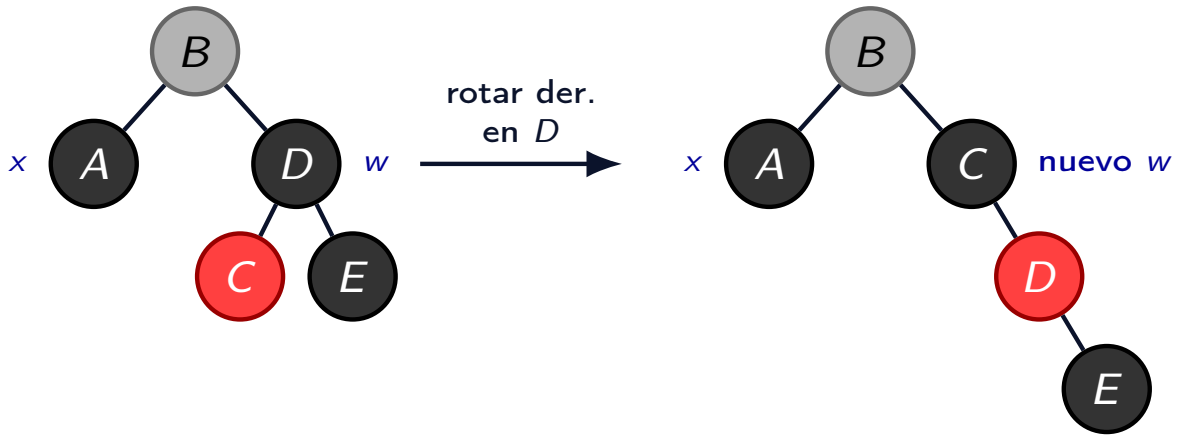
Caso 2: w negro, con sus dos hijos negros



$w.color \leftarrow \text{ROJO}, \quad x \leftarrow x.padre$

En gris, un color que no importa. Quitarle un negro a los dos lados (a x y a w) es lo mismo que quitárselo a B: el negro extra sube a B. Si B era rojo, el ciclo termina y la última línea lo pinta de negro.

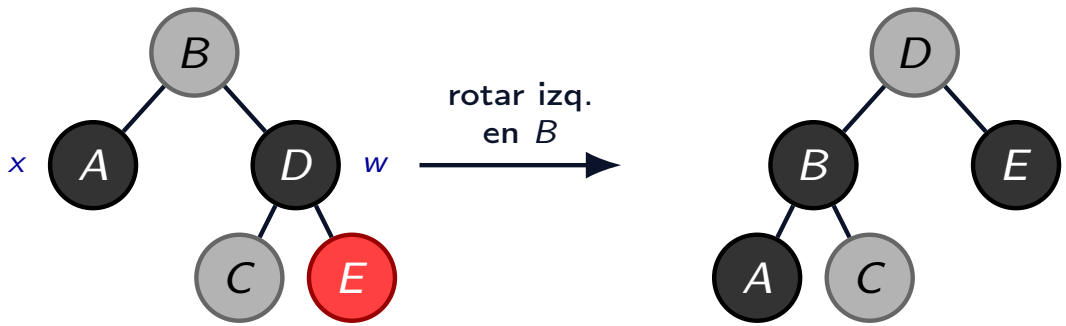
Caso 3: w negro, hijo izquierdo rojo y derecho negro



$w.izq.color \leftarrow$ NEGRO, $w.color \leftarrow$ ROJO
ROTAR-DERECHA(T, w), $w \leftarrow x.padre.der$

Tampoco resuelve todavía: deja a w con un hijo **derecho** rojo. Es el caso 4.

Caso 4: w negro, con hijo derecho rojo



$w.color \leftarrow x.padre.color$, $x.padre.color \leftarrow$ NEGRO
$w.der.color \leftarrow$ NEGRO
ROTAR-IZQUIERDA$(T, x.padre)$, $x \leftarrow T.raiz$

D sube con el color que tenía B, y el nuevo negro de B absorbe el negro extra de x. Todo queda en regla: $x \leftarrow T.raiz$ solo sirve para salir del ciclo.

RB-Eliminar-Arreglar

Algoritmo 7: RB-ELIMINAR-ARREGLAR(T, x)

```
mientras x ≠ T.raiz y x.color = NEGRO hacer
    si x = x.padre.izq entonces
        w ← x.padre.der                                   // el hermano
        si w.color = ROJO entonces caso 1
        si w.izq.color = NEGRO y w.der.color = NEGRO entonces
            caso 2
        en otro caso
            si w.der.color = NEGRO entonces caso 3
            caso 4
    en otro caso
        lo mismo, intercambiando izq y der
x.color ← NEGRO
```

¿Cuánto cuesta eliminar?

El caso 2 puede repetirse, subiendo x. ¿Y los demás?

¿Cuánto cuesta eliminar?

El caso 2 puede repetirse, subiendo x. ¿Y los demás?

$O(\lg n)$ vueltas, y a lo más 3 rotaciones

El caso 2 sube x un nivel y solo recolorea: $O(\lg n)$ veces. Los casos 1, 3 y 4 llevan al final del ciclo: tras el caso 1 viene el 2 (que termina, porque el padre quedó rojo), el 3 o el 4; tras el 3 viene el 4; y el 4 termina.

RB-Eliminar cuesta $O(\lg n)$, con a lo más 3 rotaciones. Un AVL puede necesitar $O(\lg n)$ rotaciones al eliminar: esa es la ventaja del Red-Black.

Cierre de la sesión

- El nodo guarda *color* y *padre*; el centinela $T.nil$ (negro) reemplaza a todos los NULO.
- RB-INSERTAR: el INSERTAR del BST con el nodo en rojo, y un arreglo con tres casos según el tío: recolorear y subir, o una o dos rotaciones y fin.
- RB-ELIMINAR: el ELIMINAR del BST, recordando el color de y y quién ocupa su lugar (x); si salió un negro, x carga un negro extra que se resuelve con cuatro casos según el hermano.
- Las dos operaciones cuestan $O(\lg n)$, con a lo más 2 y 3 rotaciones. Un campo como *tam* se mantiene igual que en el AVL: cada rotación actualiza los dos nodos que movió.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos por qué conviene un centinela $T.nil$ y cómo se rota con punteros al padre.
- Sabemos implementar RB-Insertar y reconocer los tres casos del arreglo.
- Sabemos implementar RB-Eliminar y explicar el negro extra y los cuatro casos del hermano.
- Sabemos contar cuántas rotaciones hace cada operación, y compararlo con el AVL.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias